

HubV3 — Human-in-the-Loop Acceptance Testing

Feature: HubV3 Hub-Centered Contextualization Intake **Branch under test:** feat/plc-mapper-gui @ 4eaa2dec (merge of #2134 — folds HubV3 P0-P6) **PRD:** docs/plans/2026-06-20-hubv3-contextualization-intake-prd.md (§6 Test Matrix, §7 Demo Acceptance) **Date:** 2026-06-21 · **Tester:** ____

· **Result:** ☐ PASS ☐ FAIL

1. Why this document exists

The HubV3 test matrix (PRD §6) has **12 acceptance items**. Eleven of them are covered by automated tests that pass green (174 tests across four lanes — see §2). **Item 12 — the end-to-end Garage Conveyor demo — cannot be fully automated:** it spans two separate stacks (the offline Windows Contextualizer and the cloud Hub), requires real source fixtures, and ends in a *human approval* that publishes the model. This is the doctrinal core of HubV3 ("Hub owns truth; approval is a human action") and therefore must be witnessed by a human.

This runbook is that witness procedure. Work top to bottom, tick each box, and capture the named evidence. A single unchecked **MUST** box = FAIL.

2. Automated coverage (the floor — do not re-test by hand)

These passed on the branch under test. They are the *foundation*; this runbook exercises only what they cannot.

Lane	Suite	Result
Python	Telegram intake (test_telegram_hub_intake , test_telegram_photo_hub_wiring , adapter, image)	✓ 29
Hub TS (vittest)	contextualization unit (intake-contract, asset-matcher, approval, bundle-import, routes)	✓ 60
Hub TS (vittest)	import integration vs live Postgres (migs 055+056)	✓ 4
Python	mira-contextualizer bundle/export suite	✓ 81
Total		✓ 174 / 0 failed

§6	Item	Automated by	Status
1	Hub accepts shared intake contract	<code>import.integration.test.ts</code> (test 1)	✓
2	Offline bundle imports as a batch	import integration + <code>bundle-import.test.ts</code>	✓
3	Same sha256 → no duplicate source	<code>import.integration.test.ts</code> (test 3)	✓
4	Strong match → stage under existing asset	<code>asset-matcher.test.ts</code> (test 4)	✓
5	No match → draft asset proposal	<code>asset-matcher.test.ts</code> (test 5)	✓
6	Probable match requires confirmation	<code>asset-matcher.test.ts</code> (test 6)	✓
7	Imports never overwrite approved data	<code>approval.test.ts</code> (test 7)	✓
8	UNS/i3X stay proposed until approved	<code>approval.test.ts</code> (test 8)	✓
9	Telegram source → same pipeline, proposed	<code>test_telegram_hub_intake</code> + wiring	✓
10	Sanitized bundle has no raw doc payloads	<code>test_sanitized_bundle_has_no_raw_document_payloads</code>	✓
11	Full bundle preserves provenance	<code>test_full_bundle_emits_evidence_and_preserves_provenance</code>	✓
12	Conveyor demo end-to-end (cross-stack)	THIS RUNBOOK	☐

3. Prerequisites

Tick each before starting. If any is missing, stop — the run is invalid.

- ☐ **Branch checked out / deployed:** Hub running `feat/plc-mapper-gui @ 4eaa2dec` (or its descendant) in a **non-prod** environment (staging or dev — never `@FactoryLM_Diagnose` / prod NeonDB).
- ☐ **Migrations applied to the target DB:** `055_contextualization.sql` **and** `056_contextualization_intake.sql` (verify via `db-inspect.yml`, never `psql prod`).
- ☐ **Authenticated Hub session** as a **UUID tenant** (slug tenants 401 since `369513cb`). Note tenant UUID: _____
- ☐ **Offline Contextualizer** built from the same branch (`mira-contextualizer/`), launchable via `python -m mira_contextualizer`.
- ☐ **Real source fixtures on disk** (PRD §7):

- ☐ MIRA/plc/Micro820_v4.1.9_Program.st
 - ☐ MIRA/plc/MbSrvConf_v4.xml
 - ☐ ~/Downloads/gs10usermanual.pdf (GS10/GS11 drive manual)
 - ☐ (optional) nameplate / wiring photos for the Telegram leg (§5)
 - ☐ **Screenshot folder ready:** every screenshot below is ALSO saved to docs/promo-screenshots/ as YYYY-MM-DD_hubv3-<step>_<viewport>.png (Screenshot Rule, append-only).
-

4. Test 12 — Garage Conveyor, offline → Hub → published

The canonical flow from PRD §7. Each step has an **action**, an **expected result**, and the **evidence** to capture. MUST = blocking.

Step 1 — Offline: create the profile

Action: In the Contextualizer, create/open profile "**Garage Demo / Micro820 Conveyor**".

Expected: A .miraprofile is created with identity fields (customer/site/area/line, controller, IP, PLC program). Empty source list.

Evidence: screenshot of the new profile screen.

- ☐ **MUST** — profile created, no error.

Step 2 — Offline: add evidence

Action: Add sources: the Micro820/CCW export (.st), the Modbus/tag config (MbSrvConf_v4.xml), and the GS10 drive manual PDF. Add nameplate/wiring photos if available.

Expected: Each source lands with a computed **sha256**; status moves pending → processing → done .

Evidence: screenshot of the source list showing per-source sha256 + done .

- ☐ **MUST** — every source reaches done ; each shows a 64-char sha256.
- ☐ no source stuck in error .

Step 3 — Offline: local parse → proposals

Action: Let the offline pipeline parse locally.

Expected: Proposed — asset identity, controller identity, signals/tags, UNS mappings, i3X projections, drive parameters, fault catalog (ISO-14224 shape), UCUM units/ranges/setpoints, scorecard. **All marked proposed/pending — nothing "approved"**.

Evidence: screenshot of the proposals/scorecard view.

- ☐ **MUST** — non-empty signals, UNS mappings, i3X objects, fault catalog, scorecard.
- ☐ **MUST** — every item shows a *proposed/pending* status (offline approval is field/pre-review only, never final).

Step 4 — Offline: export the bundle

Action: Export `machine_context_bundle.zip` (Full Evidence Bundle mode).

Expected: A deterministic zip containing `manifest.json`, `profile.json`, `sources.json`, `evidence.json`, `uns.json`, `i3x.json`, `kg_entities.json`, `kg_relationships.json`, `signals.csv`, `fault_catalog.json`, `parameters.json`, `scorecard.json`, `review.json`, `documents/*.json`, `report.md`, `IMPORT.md`.

Evidence: terminal/Finder listing of the zip contents.

- ☐ **MUST** — `evidence.json` present and non-empty (full mode carries provenance).
- ☐ `manifest.json` carries `asset_match` + `import{intent,policy:"propose_only"}` + `sources[].sha256`.

Step 5 — (optional) Telegram leg — same pipeline (\$6 test 9 live)

Action: From the Telegram bot, send a nameplate **photo** and the **GS10 PDF** with a caption naming the asset (e.g. "conveyor-1 nameplate").

Expected: Bot replies "submitted to the Hub for review." The source appears in the **same Hub import/staging queue** as the offline bundle, landing **proposed**. Uploader recorded as a numeric Telegram id; controller IP / serial preserved in `asset_hints` but scrubbed from free-text evidence.

Evidence: screenshot of the bot reply + the Hub queue row.

- ☐ Telegram source enters the same import pipeline (not a separate KB).
- ☐ lands `proposed`; no free-text IP/serial leak in the evidence block.

Step 6 — Hub: import the bundle

Action: In the Hub contextualization UI, import `machine_context_bundle.zip` into a project.

Expected: Hub creates an **import batch**, dedupes sources by sha256, and stages all proposals. Re-importing the **same** zip does **not** duplicate sources (sha256 dedup).

Evidence: screenshot of the batch + staged counts; second screenshot after a re-import showing unchanged source count.

- ☐ **MUST** — import succeeds; batch created; staged signals/faults/parameters/UNS/i3X are non-empty.
- ☐ **MUST** — re-import of the identical bundle adds **0** new source rows.

Step 7 — Hub: asset match

Action: Observe the asset-matching result for the conveyor.

Expected: **Strong** match → staged under the existing conveyor asset; **probable** → flagged needs-confirmation; **no** match → a **draft asset proposal**. (Which one depends on whether the conveyor already exists in this tenant.)

Evidence: screenshot of the match decision + reason.

- ☐ **MUST** — the match outcome is one of strong / probable / none with a stated reason (no untethered floating tags).

Step 8 — Hub: review queue

- Action:** Open the review queue for the batch.
- Expected:** Every imported proposal is listed as **pending/proposed**. UNS and i3X objects are **proposed**, not verified.
- Evidence:** screenshot of the review queue.
- ☐ **MUST** — nothing is auto-verified; UNS/i3X show `proposed` .

Step 9 — Hub: approve (the human moment)

- Action:** As an admin/technician, **approve** the batch (or selected proposals).
- Expected:** Approved context is **published** to the project model + UNS + i3X + MIRA KB; status moves `proposed` → `verified` . `approved_by` recorded.
- Evidence:** screenshot of the post-approval state showing `verified` + approver identity.
- ☐ **MUST** — approval is an explicit human action (no auto-promote).
 - ☐ **MUST** — after approval, items show `verified` with an `approved_by` .

Step 10 — Hub: no-overwrite guard

- Action:** Re-import the **same** bundle (or a slightly edited one) and approve again.
- Expected:** Already- `verified` data is **not** overwritten by the re-imported proposals; the guard reports a skip reason rather than silently clobbering.
- Evidence:** screenshot showing the prior verified values intact + a skip/no-overwrite indication.
- ☐ **MUST** — approved/verified values are unchanged after re-import.

Step 11 — Available to MIRA

- Action:** Ask MIRA (chat / direct connection) a grounded question about the conveyor that depends on the just-published context (e.g. a GS10 fault code or a tag role).
- Expected:** MIRA answers with a **cited** response grounded in the published model/manual — i.e. the approved context is live.
- Evidence:** screenshot of the grounded, cited answer.
- ☐ published context is retrievable and cited by MIRA.

5. Result & sign-off

	Count
MUST boxes total	12
MUST boxes checked	_____
Non-MUST boxes checked	_____
Screenshots captured (also in <code>docs/promo-screenshots/</code>)	_____

Overall §6 Test 12: ☐ PASS (all MUST checked) ☐ FAIL

Notes / defects observed (file as GitHub issues, link here):

Tester signature: ____ Date: __ Reviewer (Hub-as-SoR owner): __ Date: _____

6. Environment & safety reminders

- Run against **dev or staging only**. Never point this at prod NeonDB, the prod Hub, or `@FactoryLM_Diagnose` (env-boundary doctrine, `docs/environments.md`).
- MIRA is **read-only** in beta — Step 11 asks MIRA to *answer*, never to *act* on the conveyor (train-before-deploy + fieldbus-readonly rules).
- Migrations reach the target env via `apply-migrations.yml` (`dry-run` then `apply`), dev → staging → prod. Never hand-edit schema.
- Every screenshot is append-only in `docs/promo-screenshots/` — never delete.